



GREEDYGYM

SISTEMA DE GESTIÓN PARA GIMNASIOS

Rocio Baggio, Julian Fernandez, Diego Forni, Juan Loncharich

Ingeniería de Software II – 2025

Metodología de desarrollo	5
Especificación del proyecto	5
Requisitos	6
Requisitos funcionales	6
Gestión de usuarios del sistema y roles	6
Gestión de cuotas mensuales	6
Gestión de deudas	7
Campañas promocionales	7
Saludos automáticos de cumpleaños	7
Gestión y seguimiento de rutinas	7
Requisitos no funcionales	7
Entorno	8
Arquitectura	8
Herramientas utilizadas	8
Base de datos	10
Tipo de software	11
Herramienta de comunicación del equipo	11
Herramienta de modelado	12
Herramienta de prototipado	12
Herramienta de codificación	12
Diseño	12
Diagrama de casos de uso	12
Escenarios casos de uso	13
Diagrama de secuencia del sistema	13
Prototipado de pantallas	13
Análisis y diseño	13
Diagrama de clases de diseño	13
Diagrama de secuencia de diseño	13
Diagrama de paquetes de diseño	13

Diagrama entidad relación base de datos	13
Construcción	13
Implementación	13
Framework de trabajo	13
Base de Datos	13
Codificación	13
Transición	14
Propuesta de mejora	14
Referencia de mercado	14
Propuesta de mejora	14
Modificación del diagrama de clases	14
Refactorización — Vista	14
Refactorización — Controlador	14
Refactorización — Modelo	14
Refactorización — Acceso a datos	14
Material de exposición en conferencia	14
Porcentaje avance del software	14
PowerPoint presentación	14
Referencias	14
Anexo	14

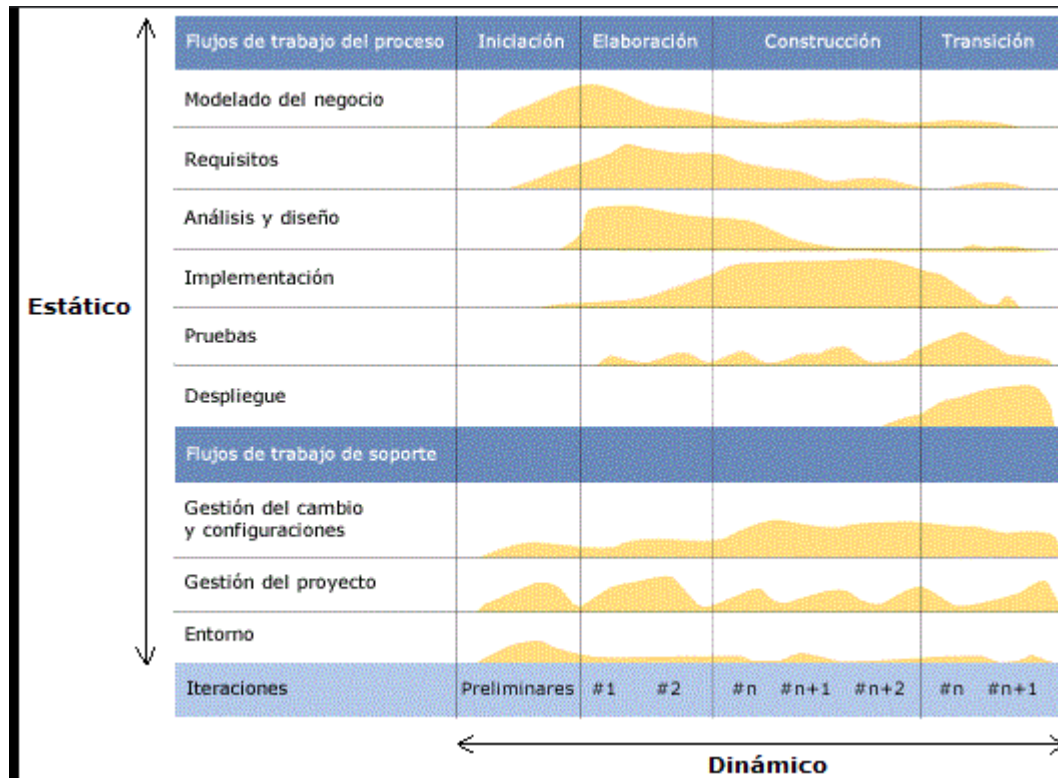
Metodología de desarrollo

Para el desarrollo de este proyecto se utilizó la metodología RUP (Rational Unified Process).

RUP es un proceso de desarrollo de software iterativo e incremental que organiza el trabajo en fases y permite gestionar cambios de manera controlada, enfocándose en la calidad del producto.

Etapas de RUP:

1. **Inicio (Inception):** Definición de la visión del sistema, identificación de actores principales y casos de uso críticos, análisis preliminar de riesgos y estimación inicial de recursos y costos.
2. **Elaboración (Elaboration):** Refinamiento de requisitos, creación del modelo de arquitectura del sistema y planificación detallada de las iteraciones futuras.
3. **Construcción (Construction):** Desarrollo completo de la funcionalidad del sistema, integración de componentes y pruebas para garantizar la calidad.
4. **Transición (Transition):** Entrega del producto al cliente, pruebas de aceptación, despliegue en el entorno real y capacitación de usuarios.



Especificación del proyecto

El equipo de desarrollo fue contratado por el Gimnasio “**Sport**” para diseñar e implementar un **software de gestión integral de asociados/as**.

El objetivo principal del sistema es optimizar y centralizar la administración de los socios y las operaciones internas del gimnasio, ofreciendo una herramienta confiable, intuitiva y eficiente.

Funcionalidades principales:

- Gestión de usuarios:** alta, consulta, modificación y baja de socios/as y usuarios del sistema, incluyendo roles y permisos.
- Gestión de cuotas:** administración de cobros, registro de pagos y vencimientos.
- Gestión de deudas:** identificación y seguimiento de cuotas impagas y alertas.
- Campañas promocionales:** creación y envío de campañas publicitarias por correo electrónico.
- Saludo de cumpleaños:** envío automático de mensajes personalizados.

F. **Gestión de rutinas:** asignación y actualización de rutinas de entrenamiento.

Requisitos

Requisitos funcionales

Gestión de usuarios del sistema y roles

- Inicio de sesión mediante nombre de usuario y contraseña
- Alta, consulta, modificación y baja lógica de usuarios
- Asignar y revocar roles y permisos

Gestión de socios/as

- Alta de socios/as con datos personales y contacto
- Consulta y búsqueda de socios/as por nombre, DNI o correo
- Modificación de datos de socios/as
- Baja lógica de socios/as preservando historial
- Validar unicidad de identificadores (DNI, correo)

Gestión de cuotas mensuales

- Alta de cuotas con sus datos (monto, periodo de vigencia, valor de cuota asociado)
- Modificación de cuotas

- Baja lógica de cuotas
- Consulta y búsqueda de cuotas
- Alta de valores de cuota
- Baja de valores de cuota
- Búsqueda y consulta de valores de cuota
- Generar obligaciones mensuales de pago
- Registrar pagos con Mercado Pago y generar comprobante

Gestión de deudas

- Identificar y listar deudas por socio/a
- Enviar recordatorios de deuda por correo electrónico

Campañas promocionales

- Permitir el envío de campañas publicitarias

Saludos automáticos de cumpleaños

- Detectar fecha de cumpleaños
- Enviar automáticamente un saludo por correo electrónico

Gestión y seguimiento de rutinas

- Profesores crean y asignan rutinas

- Actualizar/reemplazar rutinas preservando historial
- El socio/a consulta su rutina vigente

Requisitos no funcionales

- **Rendimiento:** Responder en menos de 3 segundos en operaciones comunes bajo carga normal.
 - **Usabilidad:** Interfaz intuitiva; un usuario nuevo puede operar en menos de 15 minutos.
 - **Mantenibilidad:** Arquitectura modular y documentación técnica para facilitar cambios.
-

Entorno

Arquitectura

Para el desarrollo del sistema se adoptó una **arquitectura por capas** complementada con el patrón de diseño **Modelo–Vista–Controlador (MVC)**.

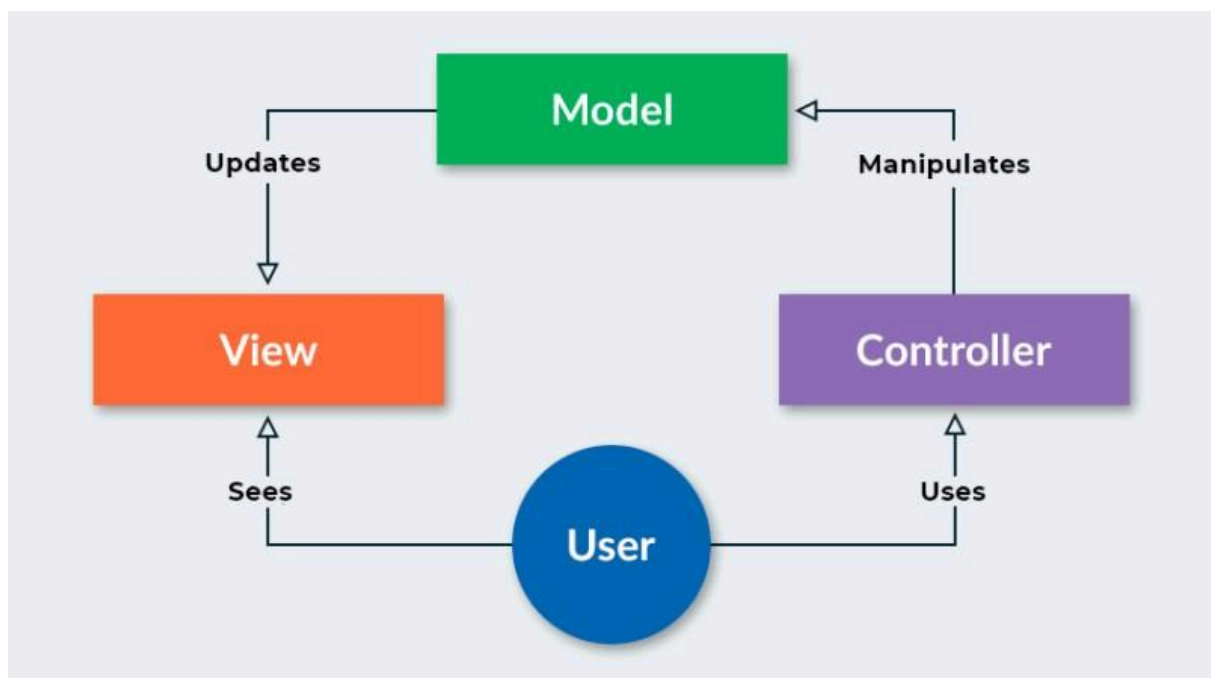
Este enfoque facilita la separación de responsabilidades, la mantenibilidad y la escalabilidad del software.

Las capas del proyecto se organizan de la siguiente manera:

- **Capa de Presentación (Vista):** Gestiona la interacción con los usuarios a través de las interfaces gráficas. Muestra información y captura datos, los cuales son enviados a la capa de control.
- **Capa de Controladores:** Actúa como intermediaria entre la vista y la lógica de negocio. Recibe las solicitudes de la capa de presentación, las valida y las delega a

las clases de servicio para su procesamiento.

- **Capa de Servicios (Lógica de Negocio):** Contiene las **clases de servicio**, responsables de implementar las reglas de negocio del gimnasio. En esta capa se procesa la información, se aplican validaciones específicas y se coordinan las operaciones necesarias (por ejemplo, el cálculo de deudas, la gestión de cuotas o la asignación de rutinas).
- **Capa de Acceso a Datos (Modelo/Repositorio):** Encargada de la persistencia de la información. Sus clases se comunican con la base de datos para crear, consultar, modificar y eliminar registros de socios/as, usuarios, pagos y rutinas.
- **Capa de Base de Datos:** Constituida por el sistema gestor de base de datos (SGBD), que garantiza la integridad, consistencia y seguridad de la información almacenada.



Cliente	Servidor			
Capa de Presentación		Capa de Negocio		
Capa Cliente	Capa Controlador	Capa Logica de Negocio	Capa Dominio	Capa Persistencia
Front End		Back End		

Herramientas utilizadas

Herramientas más importantes

- Java 17 + Maven + Spring Boot (starters web, JPA, Thymeleaf, mail, validation):**
 Conforman la base tecnológica del backend. Java es el lenguaje y runtime; Maven gestiona dependencias y el ciclo de build; Spring Boot provee la infraestructura para levantar la aplicación rápidamente y exponer controladores web, acceder a datos con JPA/Hibernate, renderizar vistas con Thymeleaf y enviar correos.
- Base de datos (MariaDB):**
 MariaDB es el motor principal en producción.
- Mercado Pago SDK:**
 Permite integrar el flujo de pagos de cuotas, crear preferencias de pago y recibir notificaciones de estado.
- Plantilla frontend (HTML/CSS/JS + Bootstrap 4 + Font Awesome + Google Fonts):**
 Define el diseño visual del sitio (public pages y dashboards) y asegura que sea responsivo y estéticamente consistente.
- Estructura MVC + Controladores REST + Autenticación por sesión:**
 Patrón de diseño que organiza la lógica en controladores, vistas y modelos; gestiona rutas, sesiones y autorización de usuarios según roles.
- Automatización y tareas programadas:**
 Uso de `@Scheduled` para envíos automáticos (p. ej. saludos de cumpleaños).

Base de datos

Para la persistencia de la información del sistema se utilizó el motor de base de datos **MySQL/MariaDB**, en la versión:

```
mysql Ver 15.1 Distrib 10.3.39-MariaDB, for debian-linux-gnu  
(aarch64) using readline 5.2
```

La elección de este motor se fundamenta en su estabilidad, amplio soporte en entornos de producción y compatibilidad con las herramientas de desarrollo utilizadas (Spring Boot y JPA/Hibernate).

Cabe destacar que durante el desarrollo no se empleó ningún editor gráfico como MySQL Workbench. En su lugar, todas las operaciones se realizaron directamente mediante **comandos en la terminal**. Esta decisión se tomó con el fin de evitar la incorporación y el aprendizaje de una nueva herramienta externa, manteniendo el flujo de trabajo más ligero y cercano a la línea de comandos del sistema operativo.

Tipo de software

El sistema desarrollado corresponde a una **aplicación web**. Este tipo de aplicación permite el acceso desde cualquier dispositivo con navegador y conexión a Internet, sin necesidad de instalar software adicional en los equipos de los usuarios.

- **Backend:** Implementado en **Java 17** utilizando el framework **Spring Boot**. Esta capa se encarga de la lógica de negocio, la gestión de usuarios, cuotas, rutinas y la comunicación con la base de datos.
- **Frontend:** Desarrollado con **HTML, CSS y JavaScript**, apoyado en plantillas basadas en Bootstrap y Thymeleaf para la renderización de páginas dinámicas. Esta capa provee la interfaz gráfica que utilizan los distintos perfiles de usuario.
- **Comunicación entre capas:** Se realiza mediante el patrón **Modelo–Vista–Controlador (MVC)** y controladores REST de Spring, que exponen endpoints para el intercambio de datos entre cliente y servidor.

La elección de este enfoque permite mantener una clara separación entre la presentación, la lógica de negocio y el acceso a datos, logrando un sistema escalable, mantenible y de fácil uso para los usuarios finales.

Herramientas de comunicación del equipo

Para la coordinación del trabajo y el intercambio de información se utilizaron distintas herramientas de comunicación y colaboración:

- **Discord:** Se creó un canal exclusivo para el equipo de desarrollo, donde se realizaron reuniones rápidas, discusiones técnicas y coordinación de tareas.
- **GitHub:** Además de servir como repositorio de código, fue utilizado como medio de comunicación para gestionar versiones, reportar problemas y documentar cambios realizados en el proyecto.
- **WhatsApp:** Se contó con un grupo en el cual participaba también el profesor responsable de la materia. Este espacio se utilizó para resolver dudas, compartir material de apoyo y realizar consultas rápidas



Herramienta de modelado

Para la construcción y visualización de los distintos diagramas del proyecto se emplearon herramientas específicas de modelado:

- **Draw.io:** Utilizada para la elaboración de los **diagramas de secuencia**, **diagramas de caso de uso** y **escenarios de casos de uso**. Esta herramienta permitió diseñar diagramas de manera colaborativa y exportarlos en formatos compatibles para la

documentación.

- **UMLet:** Herramienta utilizada para abrir y trabajar con el **diagrama de clases** provisto por el profesor en formato **.uxf**. Facilitó la visualización y comprensión de la estructura de clases definida como base del sistema
- **Mermaid Chart:** En un principio se había considerado esta herramienta para la elaboración de los diagramas de clases. Sin embargo, al no ser completamente gratuita y resultar poco práctica de utilizar, se decidió emplearla únicamente para la creación del diagrama de clases de dominio.

Herramienta de prototipado

Para realizar los prototipos de pantalla se utilizó la herramienta Balsamiq.



Herramienta de codificación

Para el desarrollo del proyecto se utilizaron las siguientes herramientas de codificación:

- **Visual Studio Code (VS Code):** IDE principal utilizado por el equipo para la edición de código, integración con GitHub y ejecución de pruebas. Su ligereza y amplia disponibilidad de extensiones facilitaron el flujo de trabajo.
- **Oracle Virtual Machine:** Se utilizó una máquina virtual provista por Oracle para simplificar el desarrollo y codificación del proyecto. Esto permitió que todo el equipo de desarrollo trabajara de manera simultánea sobre el proyecto y, además, facilitó el despliegue de la aplicación.

- **GitHub Copilot:** Asistente de código basado en inteligencia artificial que permitió acelerar la escritura de código y mejorar la productividad de los desarrolladores.
- **OpenAI Codex:** Asistente de programación utilizado para obtener sugerencias, generar ejemplos de código y resolver dudas técnicas durante el desarrollo del backend y el frontend.

Diseño

Diagrama de clases de dominio

Aquí se presenta el diagrama de clases de dominio. Es un primer acercamiento que tiene como objetivo facilitar la comprensión del problema y representar de manera fiel los elementos y relaciones del mundo real.

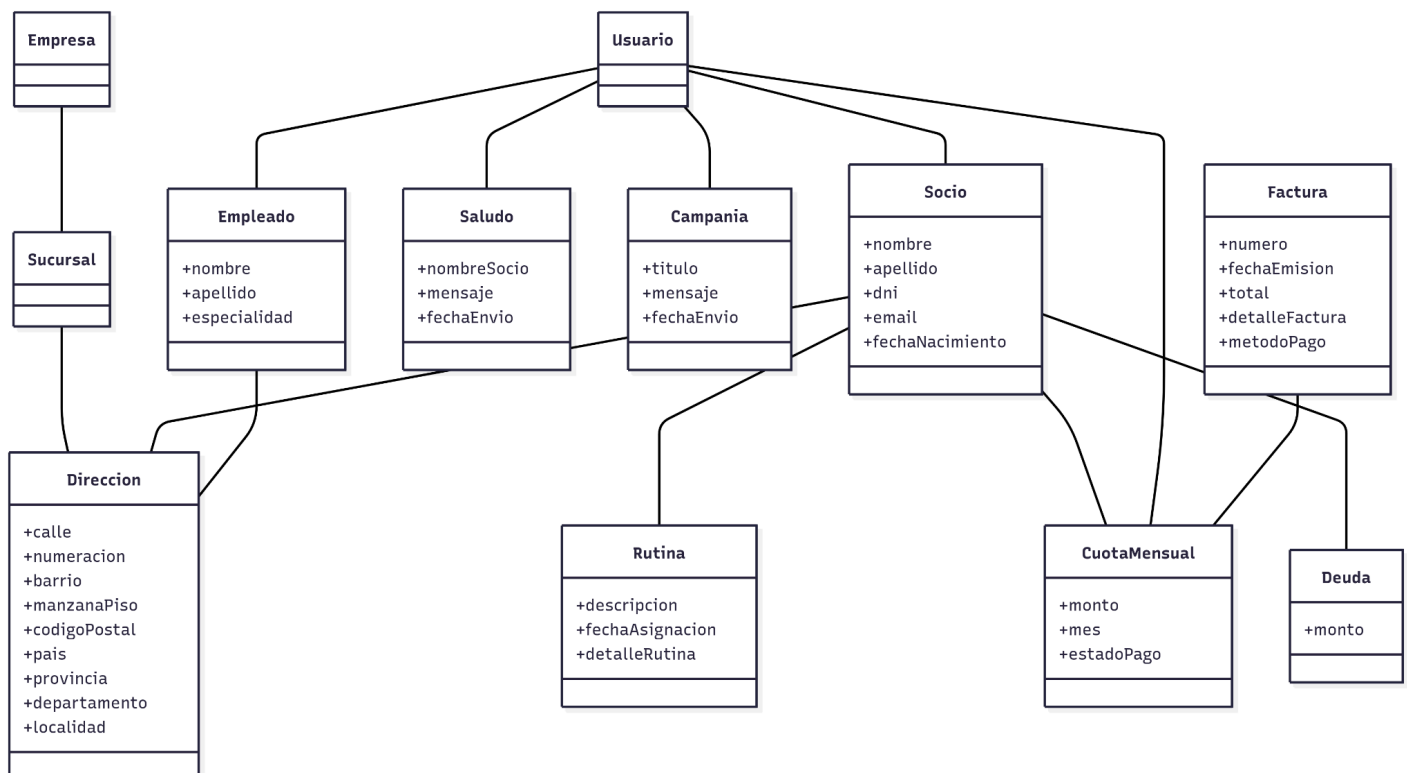


Diagrama de clases de diseño

El diagrama de clases fue consensuado junto con el profesor y modelado en UML. En él se representan las principales entidades del sistema y sus relaciones, lo que permite visualizar la estructura y organización del software. Entre las clases más relevantes se encuentran **Socio**, **Cuota Mensual** y **Valor Cuota**, que reflejan directamente los procesos centrales de gestión del gimnasio, como la administración de socios, el control de pagos y la definición de los montos de las cuotas.

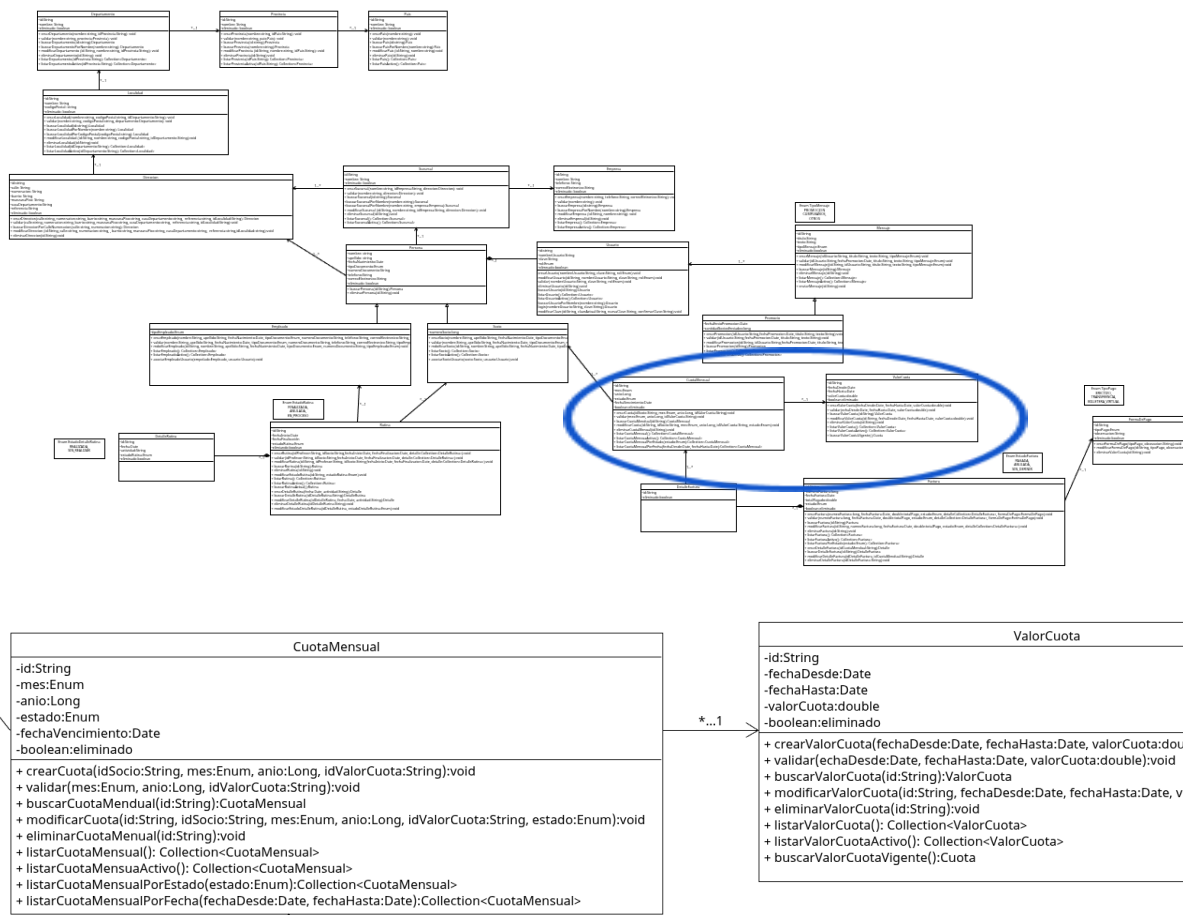


Diagrama de paquetes

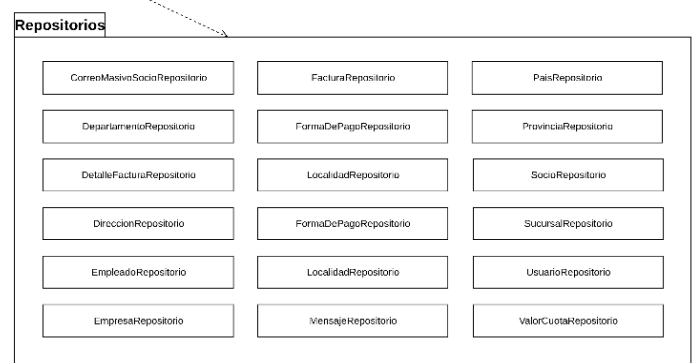
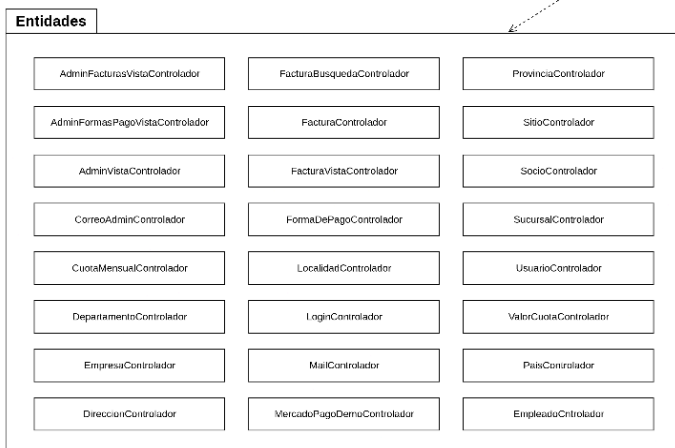
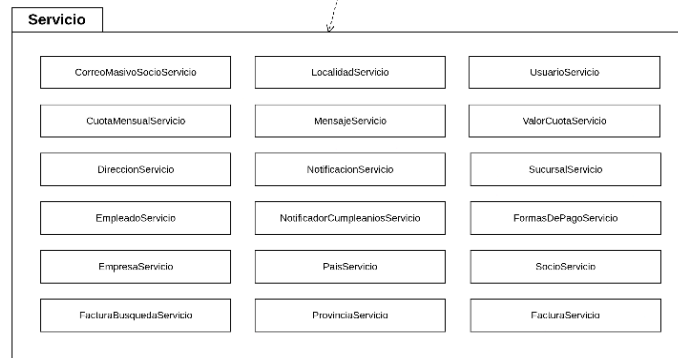
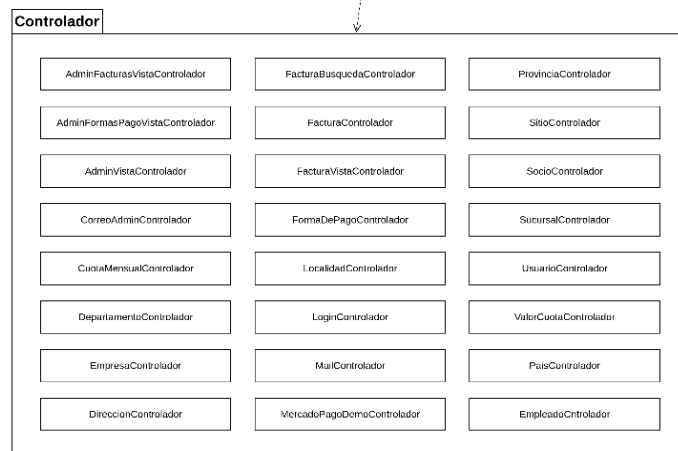
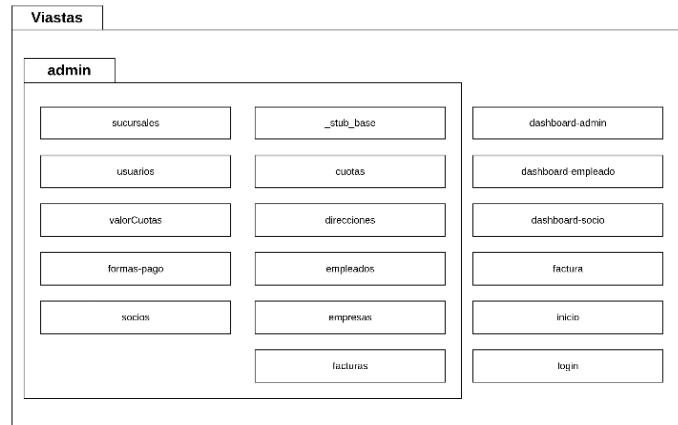
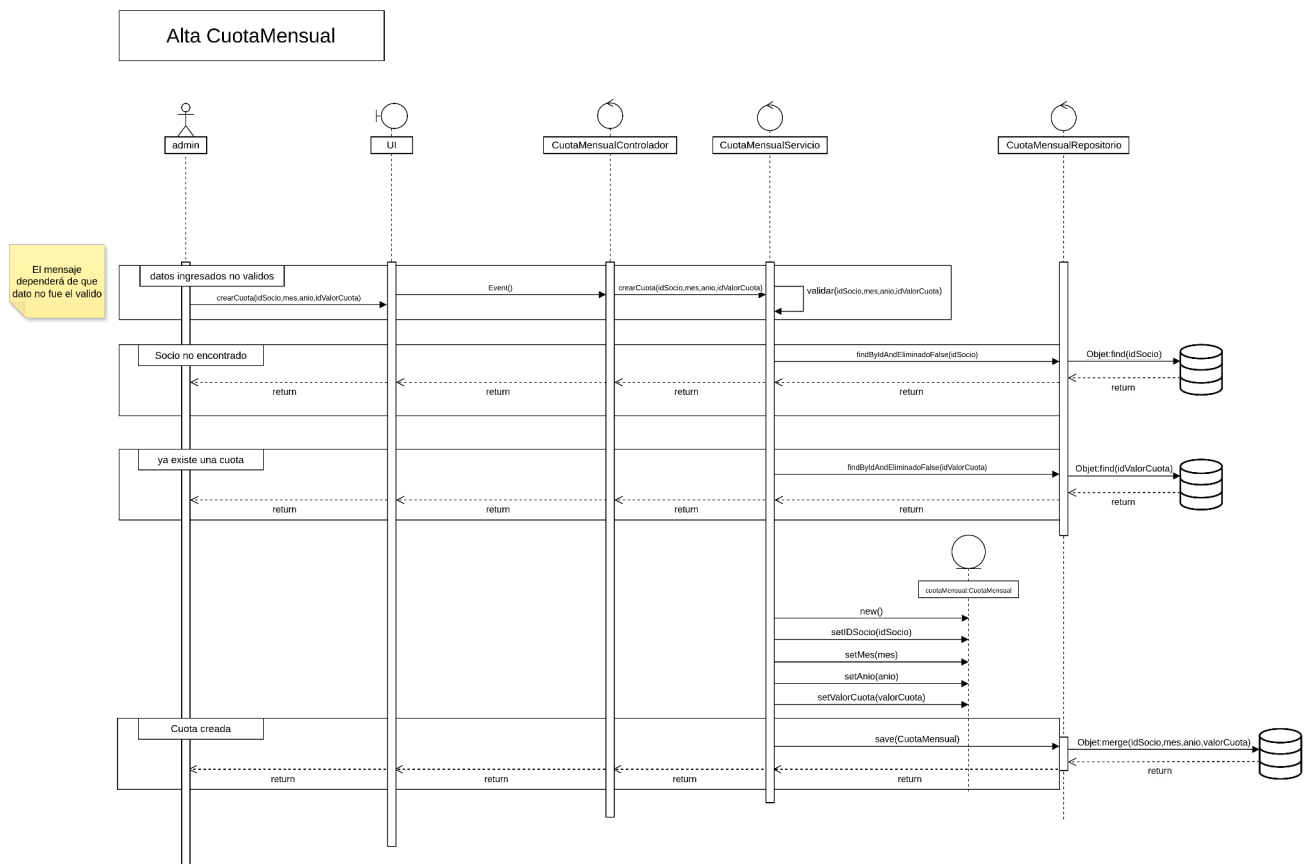


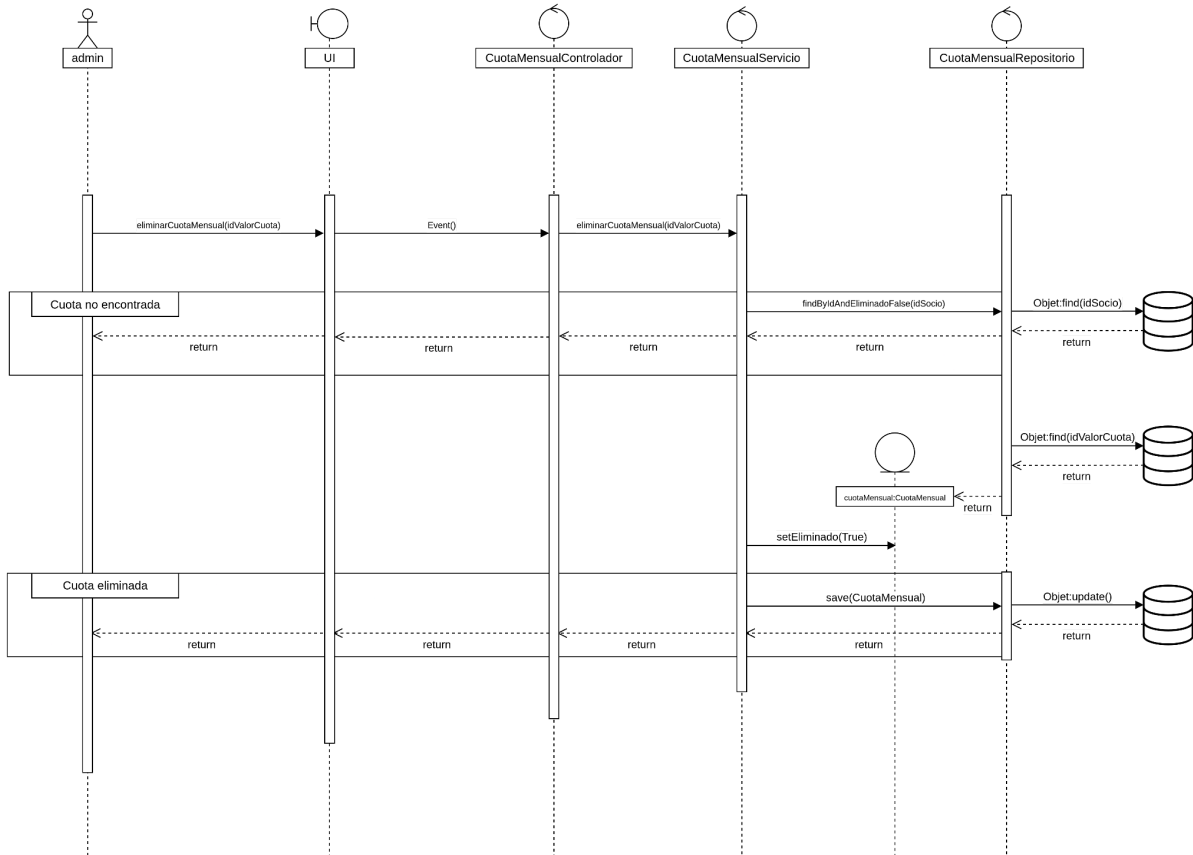
Diagrama de secuencia de diseño

Los diagramas de secuencia de diseño nos permiten visualizar la interacción entre las distintas partes del sistema (objetos, etc). Se enfoca no solo en representar la estructura interna de los componentes del software, sino que también de la lógica interna de los procesos. Estos diagramas fueron realizados únicamente para los casos de uso que les fueron asignados al equipo. A continuación se detalla el diagrama de secuencia de *ABM Cuota Mensual* y *ABM Valor Cuota*.

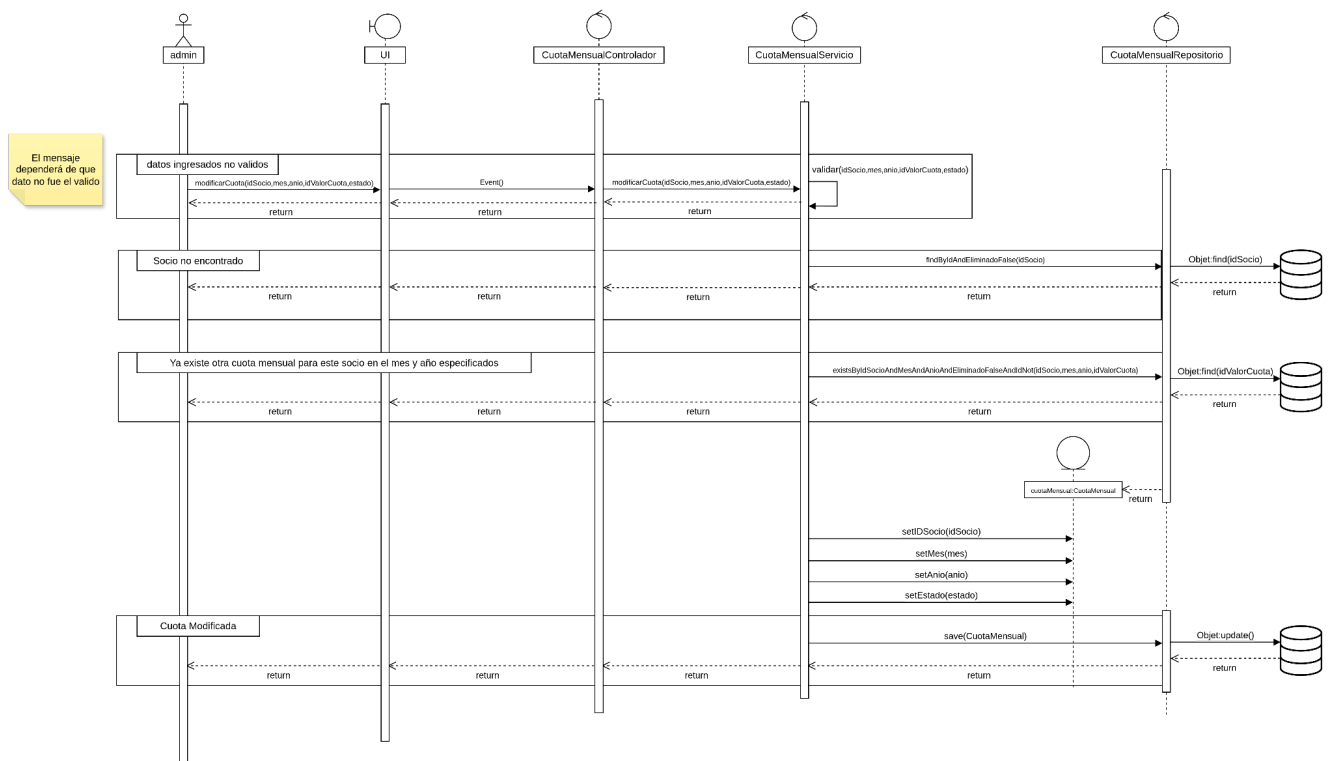
ABM Cuota Mensual



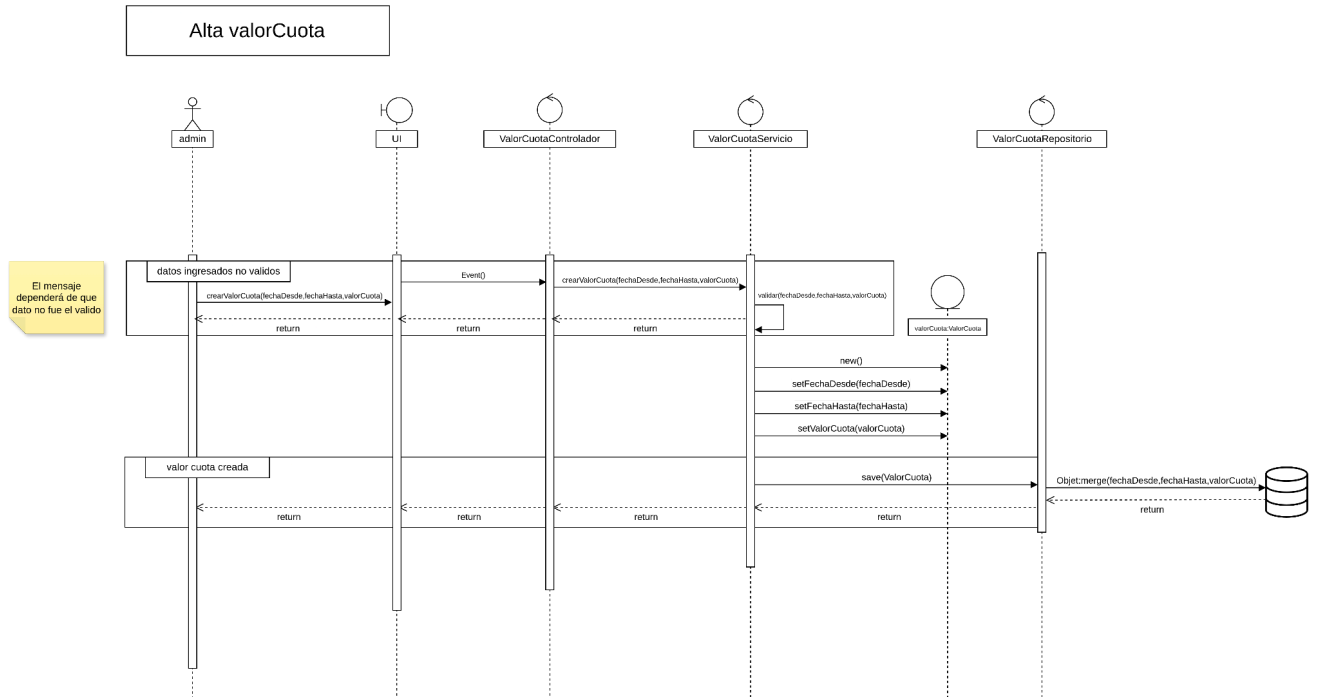
Baja CuotaMensual



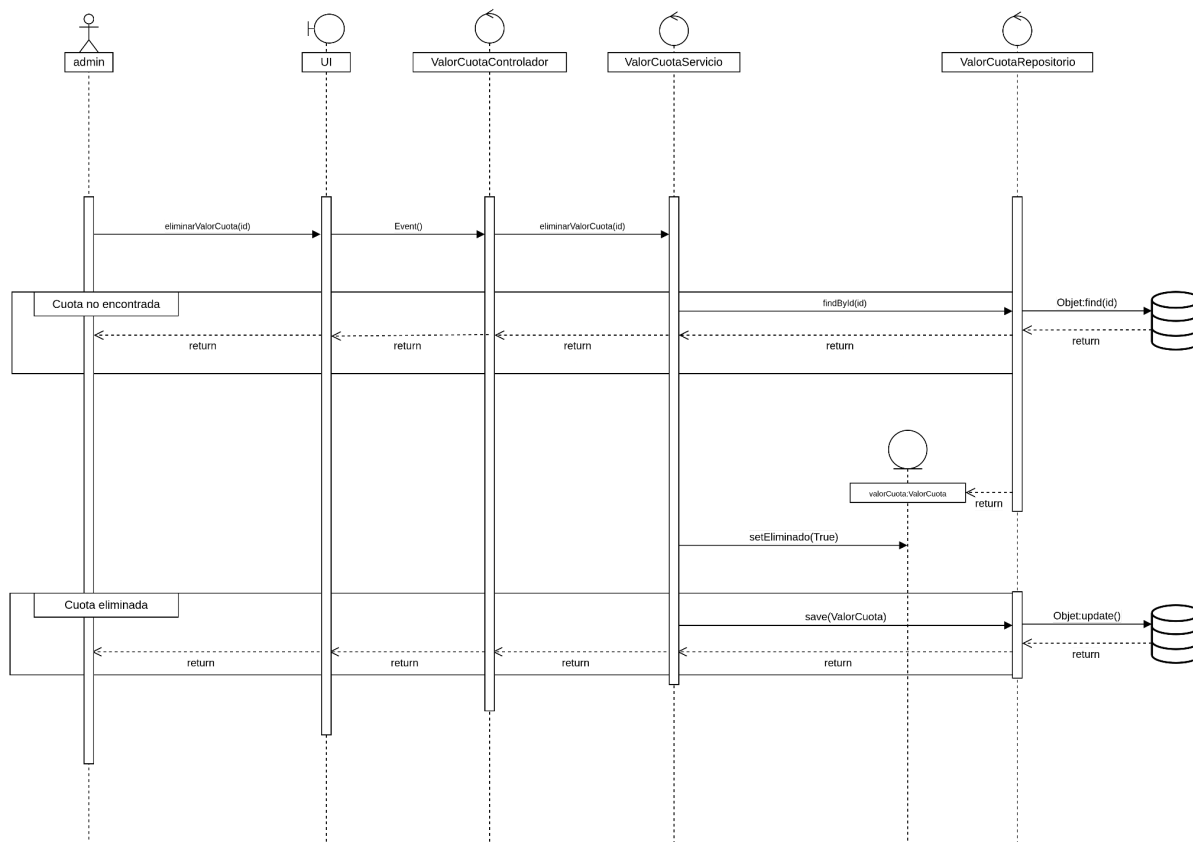
Modificacion CuotaMensual



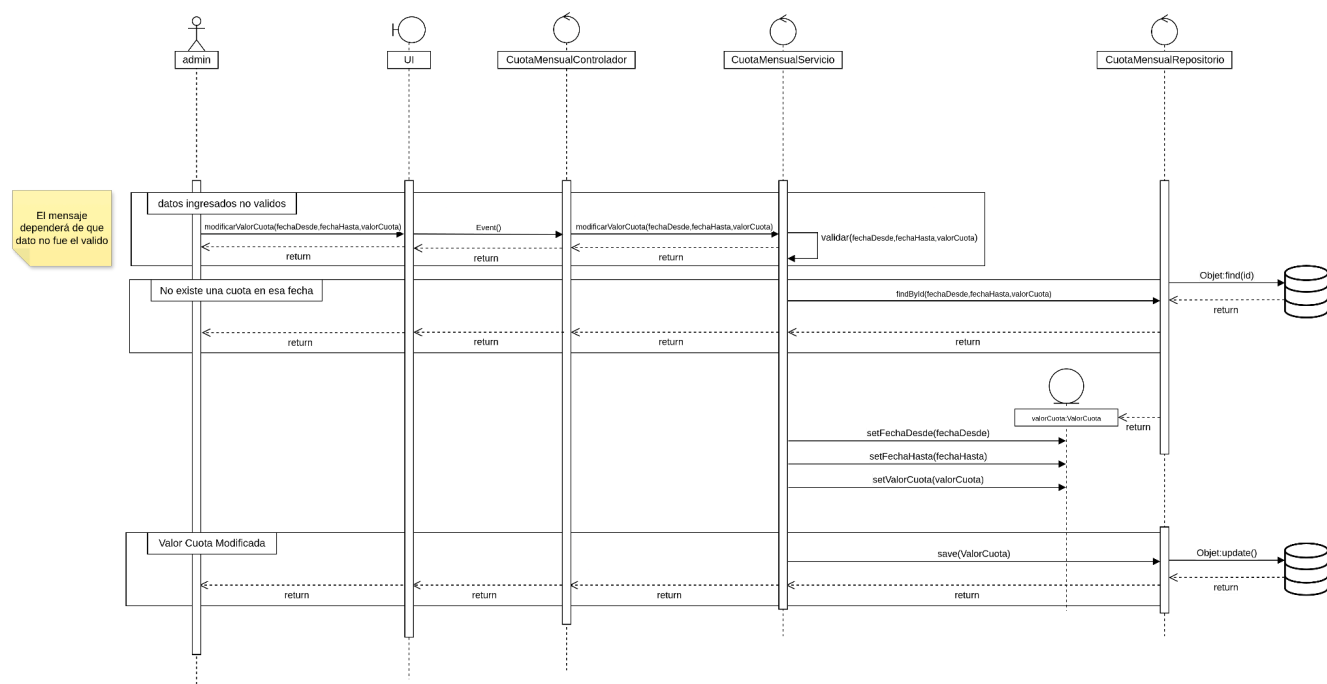
ABM Valor Cuota



Baja valorCuota



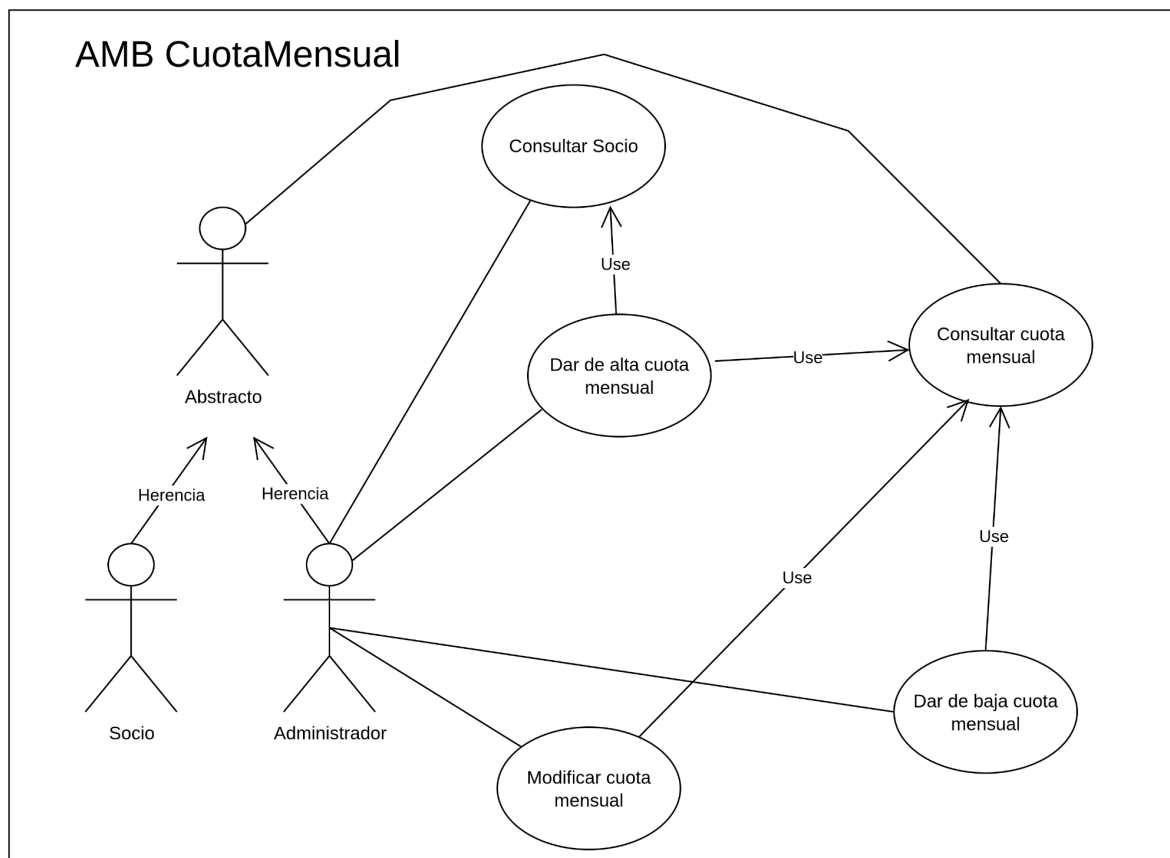
Modificacion valorCuota

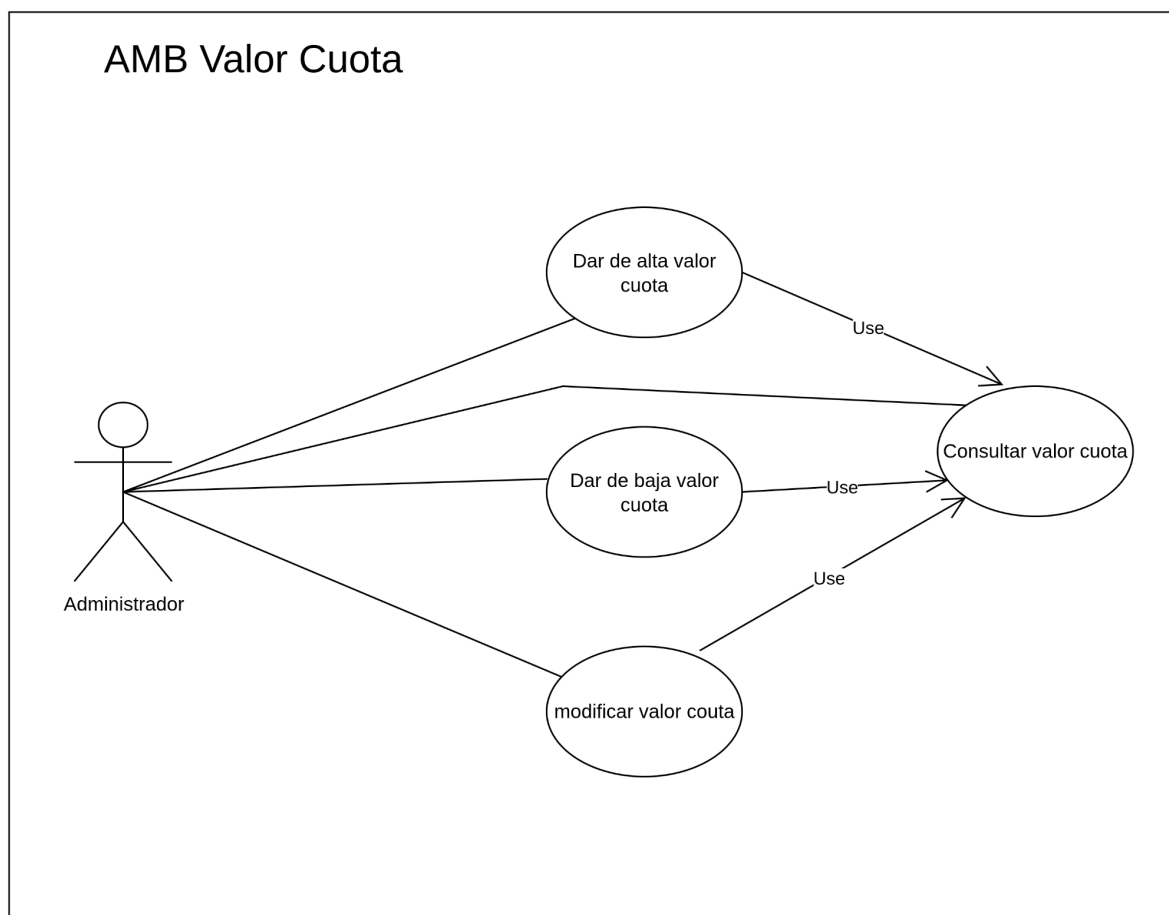


El mensaje dependerá de que dato no fue el valido

Diagrama de casos de uso

A continuación, se presentan los diagramas de casos de uso correspondientes al ABM de cuota mensual y al ABM de valor de cuota.





Escenarios casos de uso

A continuación, se presentan los escenarios de casos de uso del ABM de cuota mensual y del ABM de valor de cuota, detallando un escenario específico para cada operación de alta, baja y modificación.

ABM Cuota Mensual

Caso de Uso	Dar de baja cuota mensual
Descripción del caso de uso	Se da de baja la cuota mensual

Activación	El administrador accede la módulo de gestión de cuotas mensuales
------------	--

Flujo Principal
1. El usuario presiona el botón "Cuotas" 2. El usuario buscar en la lista la cuota que desea dar de baja 3. El usuario presiona el botón "borrar" correspondiente a la cuota que desea dar de baja 4. El sistema muestra un mensaje para confirmar la decisión del usuario de dar de baja esa cuota. El mensaje dice: "¿Estás seguro que querés eliminar esta cuota ?" 5. El usuario confirma la acción presionando el botón "aceptar" 6. El sistema valida la existencia de la cuota 7. El sistema ejecuta la baja lógica 8. El sistema muestra un mensaje de confirmación indicando que la cuota se ha eliminado correctamente

Flujos Alternativos
Alternativa al paso 5 : El usuario decide no llevar a cabo la acción y presiona el botón "cancelar"
1. El sistema regresa al panel de gestión de cuotas Fin del caso de uso

Precondiciones	
1	El administrador está autenticado en el sistema
2	Existe al menos una cuota registrada en el sistema
3	La cuota no está eliminada en el sistema

Postcondiciones	
1	La cuota queda eliminada y no se muestra en la lista de cuotas
2	La cuota ya no está disponible para su modificación
3	La baja es lógica, por lo que sigue existiendo en la base datos

Caso de Uso	Dar de alta cuota mensual
Descripción del caso de uso	Se da de alta la cuota mensual

Activación	El administrador accede la módulo de gestión de cuotas mensuales
------------	--

Flujo Principal
1. El usuario presiona el botón "Cuotas" 2. El usuario ingresa el DNI del socio a quién se le creará la cuota 3. El usuario selecciona un mes del año a través de una lista desplegable 4. El usuario ingresa el año 5. El usuario selecciona el valor de la cuota a través de una lista desplegable 6. El usuario presiona el botón "Guardar" 7. El sistema verifica que no exista otra cuota mensual para el mismo socio en el mismo mes y año 8. El sistema crea la cuota mensual 9. El sistema persiste la cuota en la base de datos 10. El sistema muestra un mensaje confirmación al usuario

Flujos Alternativos
Alternativa al paso 2 : El usuario ingresa DNI nulo
1. El sistema indica la obligatoriedad de ingresar un DNI no nulo
Alternativa al paso 2 : El usuario ingresa DNI de un socio que no existe
1. El sistema indica la obligatoriedad de ingresar un DNI de un socio existente dentro del sistema
Alternativa al paso 6: El usuario presiona el botón "cancelar"
1. El sistema vuelve automáticamente a la panel de gestión de cuota mensual
Alternativa al paso 8 : Ya existe una cuota para ese socio en ese mismo período
1. El sistema detecta que ya existe una cuota para ese socio en el año y mes especificados 2. El sistema muestra el mensaje: "Ya existe una cuota mensual para este socio en el mes y año especificados" Fin del caso de uso

Precondiciones	
1	El administrador está autenticado en el sistema
2	Existen socios en el sistema
3	Existen valores de cuota disponibles

Postcondiciones	
1	La cuota queda registrada en el sistema con estado "pendiente"
2	El socio tiene una nueva cuota asignada para el período especificado
3	La cuota queda disponible para su consulta, modificación o baja

Caso de Uso	Modificar cuota mensual
Descripción del caso de uso	Se modifica cuota mensual

Activación	El administrador accede la módulo de gestión de cuotas mensuales
------------	--

Flujo Principal
1. El usuario presiona el botón "Cuotas" 2. El usuario buscar en la lista la cuota que desea modificar 3. El usuario presiona el botón "editar" correspondiente a la cuota que desea modificar. 4. El sistema muestra una pantalla con los campos de cuota 5. El usuario modifica los campos 6. E usuario presiona el botón "guardar" 7. El sistema verifica que no exista otra cuota mensual para el mismo socio en el mismo mes y año 8. El sistema actualiza la cuota mensual 9. El sistema persiste la cuota en la base de datos 10. El sistema muestra un mensaje confirmación al usuario

Flujos Alternativos
Alternativa al paso 6 : El usuario decide no llevar a cabo la acción y presiona el botón "cancelar"
1. El sistema regresa al panel de gestión de cuotas Fin del caso de uso

Precondiciones	
1	El administrador está autenticado en el sistema
2	Existen socios y valores de cuota disponibles en el sistema
3	La cuota a modificar no está eliminada en el sistema

Postcondiciones	
1	La cuota mensual queda actualizada con los nuevos datos
2	La cuota está disponible para su consulta y modificación

ABM Valor Cuota

Caso de Uso	Modificar valor de cuota
Descripción del caso de uso	Se da de baja el valor de la cuota

Activación	El administrador accede la módulo de gestión de valor de cuota
------------	--

Flujo Principal
1. El usuario presiona el botón "Valores de cuota" 2. El usuario buscar en la lista el valor de cuota que desea modificar 3. El usuario presiona el botón "editar" correspondiente al valor de cuota que desea modificar. 4. El sistema muestra una pantalla con los campos de valor de cuota 5. El usuario modifica uno o más campos 6. E usuario presiona el botón "guardar cambios" 7. El sistema verifica la consistencia de las fechas 8. El sistema verifica que el monto ingresado sea válido 9. El sistema actualiza el valor de cuota 10. El sistema persiste el valor de cuota en la base de datos 11. El sistema muestra un mensaje confirmación al usuario

Flujos Alternativos
Alternativa al paso 6 : El usuario decide no llevar a cabo la acción y presiona el botón "cancelar"
1. El sistema regresa al panel de gestión de cuotas Fin del caso de uso
Alternativa al paso 8 A : El sistema detecta que la fecha de fin es anterior a la fecha de inicio
1. El sistema muestra un mensaje que indica que la fecha de fin no puede ser anterior a la fecha de inicio 2. El sistema muestra al usuario la interfaz para completar el campo faltante (fecha de inicio)
Alternativa al paso 8 B: El sistema detecta que el monto ingresado es nulo
1. El sistema indica que debe completarse el campo con un monto válido 2. El sistema muestra los campos a completar de valor de cuota Fin de caso de uso.

Precondiciones	
1	El administrador está autenticado en el sistema
2	Existe valores de cuota disponibles en el sistema
3	El valor de cuota a modificar no está eliminada en el sistema

Postcondiciones	
1	El valor de cuota queda actualizado con los nuevos valores
2	El valor de cuota queda disponible para su consulta

Caso de Uso	Dar de baja valor de cuota
Descripción del caso de uso	Se da de baja el valor de la cuota

Activación	El administrador accede la módulo de gestión de valor de cuota
------------	--

Flujo Principal
1. El usuario presiona el botón "Valores de cuota" 2. El usuario buscar en la lista el valor de cuota que desea dar de baja 3. El usuario presiona el botón "borrar" correspondiente al valor de cuota que desea dar de baja 4. El sistema muestra un mensaje para confirmar la decisión del usuario de dar de baja esa cuota. El mensaje dice: "¿Estás seguro que querés eliminar este valor de cuota ?" 5. El usuario confirma la acción presionando el botón "aceptar" 6. El sistema valida la existencia del valor de cuota 7. El sistema ejecuta la baja lógica 8. El sistema muestra un mensaje de confirmación indicando que la cuota se ha eliminado correctamente

Flujos Alternativos
Alternativa al paso 5 : El usuario decide no llevar a cabo la acción y presiona el botón "cancelar"
1. El sistema regresa al panel de gestión de cuotas Fin del caso de uso

Precondiciones	
1	El administrador está autenticado en el sistema
2	Existe al menos un valor de cuota registrado en el
3	El valor de cuota no está eliminado en el sistema

Postcondiciones	
1	El valor de cuota queda eliminado y no se muestra en la lista de valores de cuota activos
2	El valor de cuota ya no está disponible para su modificación
3	La baja es lógica, por lo que sigue existiendo en la base datos

Caso de Uso	Modificar valor de cuota
Descripción del caso de uso	Se da de baja el valor de la cuota

Activación	El administrador accede la módulo de gestión de valor de cuota
------------	--

Flujo Principal
1. El usuario presiona el botón "Valores de cuota" 2. El usuario buscar en la lista el valor de cuota que desea modificar 3. El usuario presiona el botón "editar" correspondiente al valor de cuota que desea modificar. 4. El sistema muestra una pantalla con los campos de valor de cuota 5. El usuario modifica uno o más campos 6. El usuario presiona el botón "guardar cambios" 7. El sistema verifica la consistencia de las fechas 8. El sistema verifica que el monto ingresado sea válido 9. El sistema actualiza el valor de cuota 10. El sistema persiste el valor de cuota en la base de datos 11. El sistema muestra un mensaje confirmación al usuario

Flujos Alternativos
Alternativa al paso 6 : El usuario decide no llevar a cabo la acción y presiona el botón "cancelar"
1. El sistema regresa al panel de gestión de cuotas Fin del caso de uso
Alternativa al paso 8 A : El sistema detecta que la fecha de fin es anterior a la fecha de inicio
1. El sistema muestra un mensaje que indica que la fecha de fin no puede ser anterior a la fecha de inicio 2. El sistema muestra al usuario la interfaz para completar el campo faltante (fecha de inicio)
Alternativa al paso 8 B: El sistema detecta que el monto ingresado es nulo
1. El sistema indica que debe completarse el campo con un monto válido 2. El sistema muestra los campos a completar de valor de cuota Fin de caso de uso.

Precondiciones	
1	El administrador está autenticado en el sistema
2	Existe valores de cuota disponibles en el sistema
3	El valor de cuota a modificar no está eliminada en el sistema

Postcondiciones	
1	El valor de cuota queda actualizado con los nuevos valores
2	El valor de cuota queda disponible para su consulta

Diagrama de secuencia de dominio

Diagrama secuencia de dominio
ABM CuotaMensual

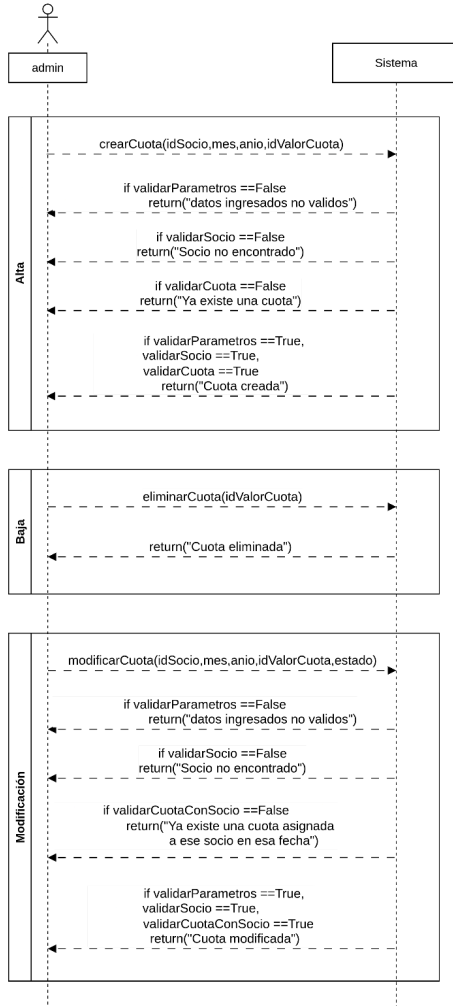


Diagrama secuencia de dominio
ABM valorCuota

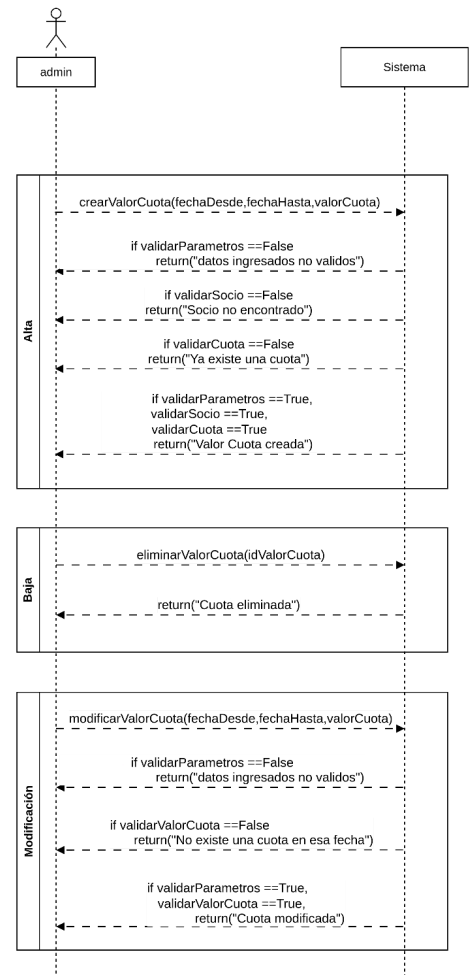
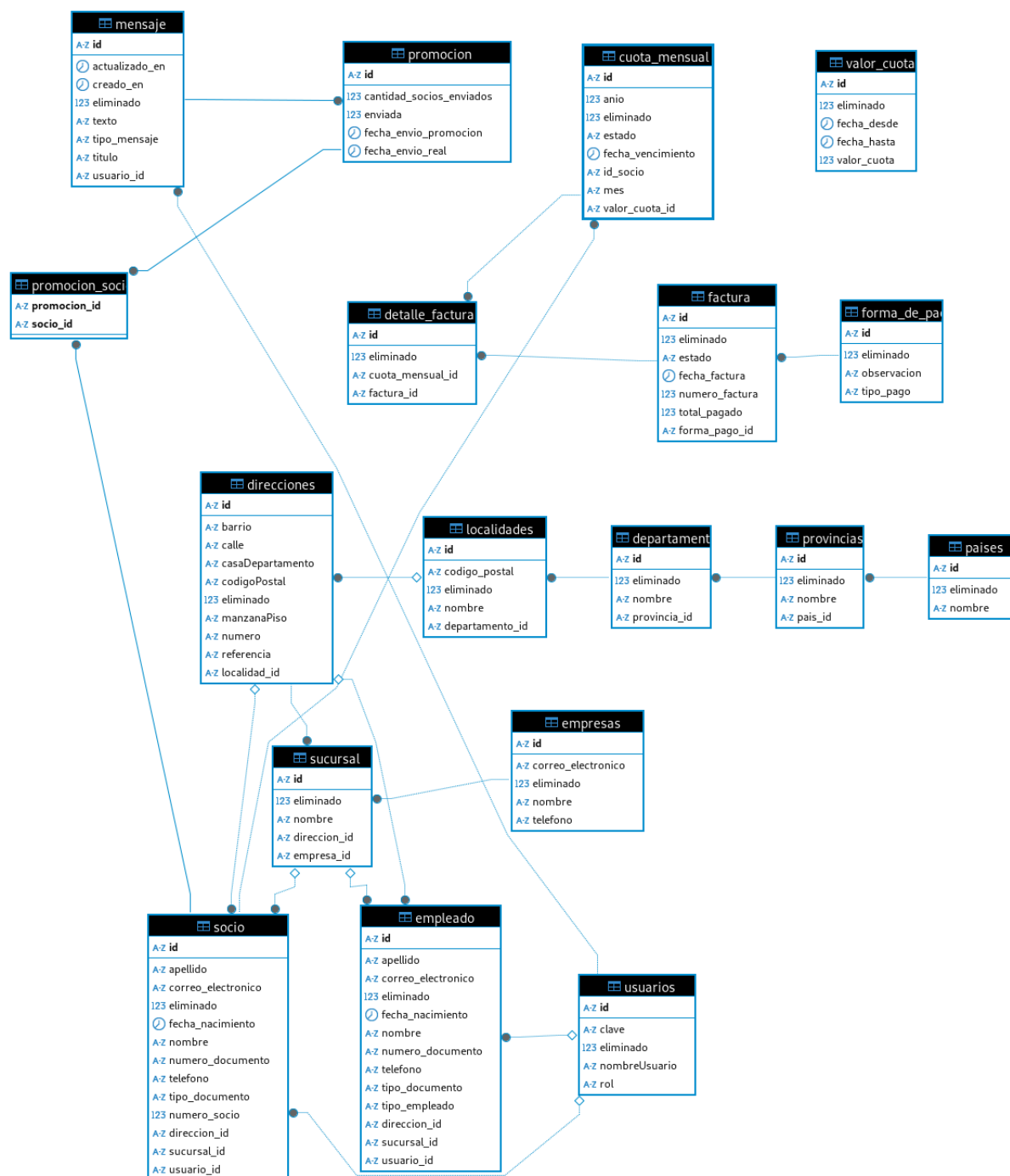
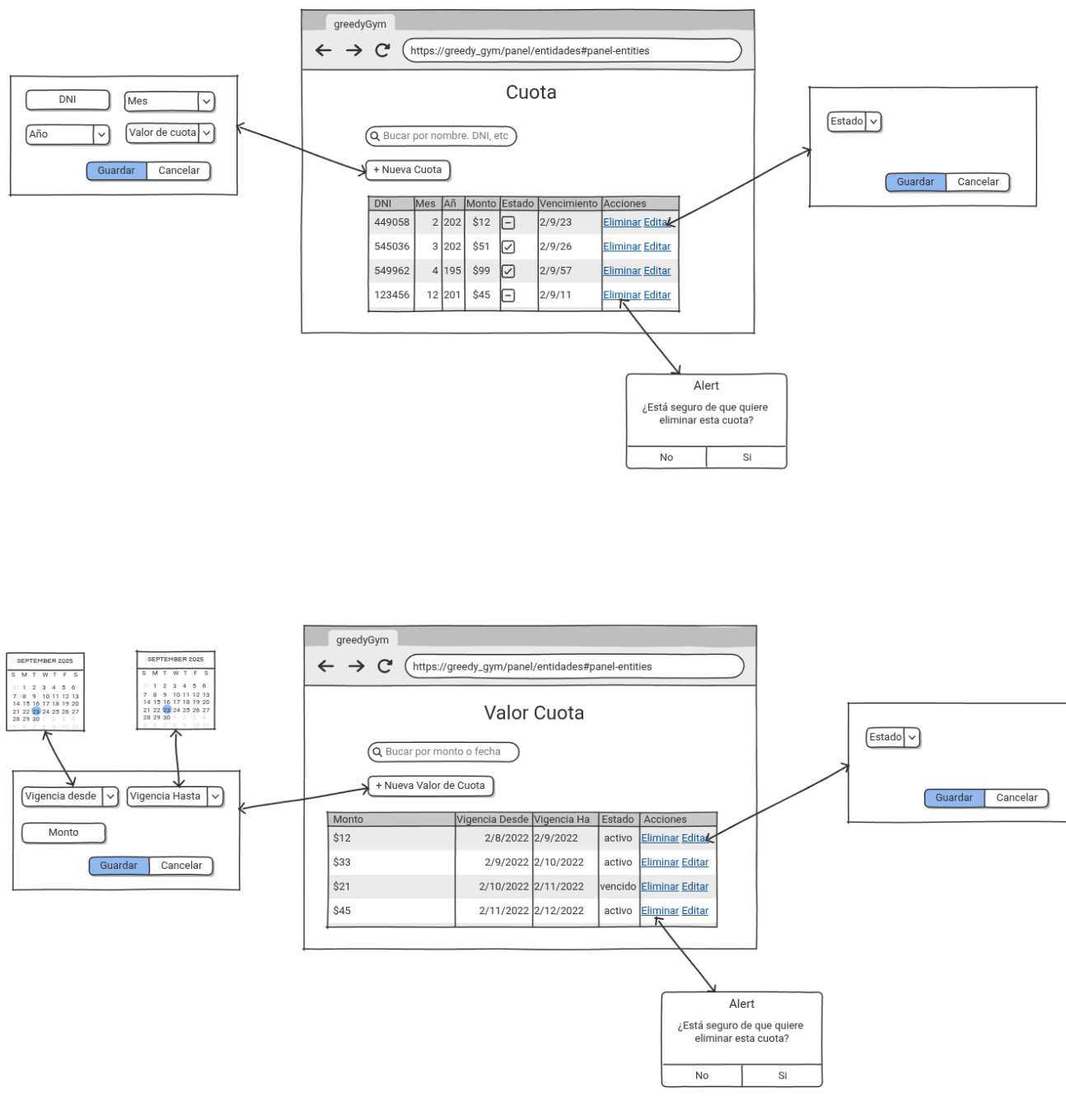


Diagrama entidad relación base de datos



Prototipado de pantallas



Construcción

La etapa de construcción se centró principalmente en la codificación del sistema. En esta fase se implementaron las distintas partes diseñadas en las etapas previas, transformando los requisitos y modelos en código ejecutable. La programación constituyó la actividad

principal y permitió obtener incrementos funcionales que consolidaron el avance del proyecto.

Implementación

En esta fase todos los integrantes del grupo comenzaron a programar siguiendo el patrón de diseño acordado (MVC). Para la colaboración se utilizaron plataformas como Git y GitHub, lo que permitió mantener el control de versiones y el trabajo en equipo. Además, se contó con una máquina virtual provista por Oracle, la cual facilitó tanto el proceso de codificación como las pruebas y el despliegue de la aplicación.

Framework de trabajo

Se empleó el framework **Spring Boot** para el desarrollo del backend, ya que simplifica la configuración y permite una integración ágil con la base de datos. En el frontend se utilizaron tecnologías como **HTML, CSS y JavaScript**, logrando una aplicación web funcional.

Base de Datos

Para la gestión de los datos se utilizó **MySQL/MariaDB** como motor de base de datos, el cual permitió almacenar y consultar de manera eficiente la información del sistema. Se diseñó el esquema de tablas en base a los requerimientos relevados en la etapa de análisis.

Codificación

La codificación se realizó de manera colaborativa, aplicando el patrón MVC. Cada integrante del equipo se enfocó en módulos específicos, lo que permitió un desarrollo paralelo y ordenado.

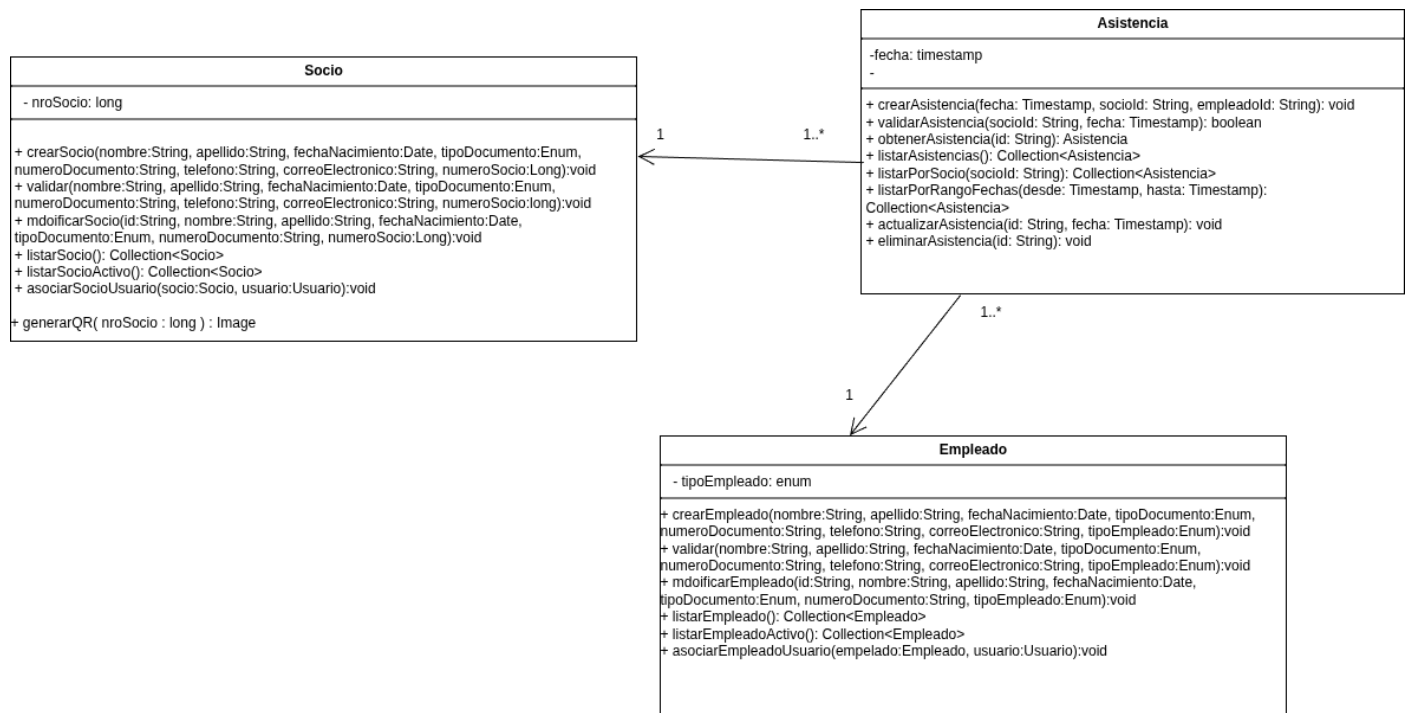
Propuesta de mejora

Registro de asistencia

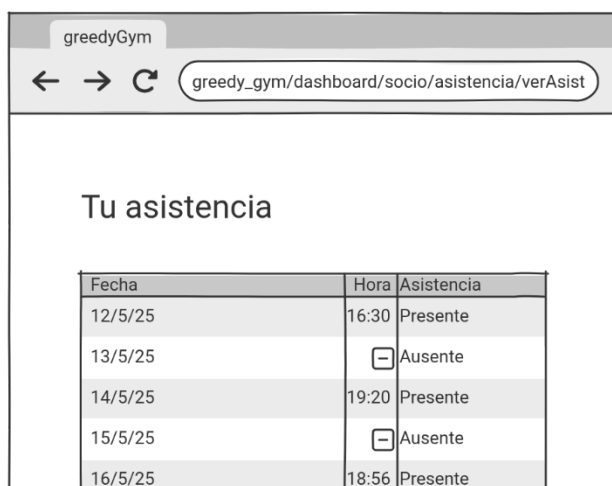
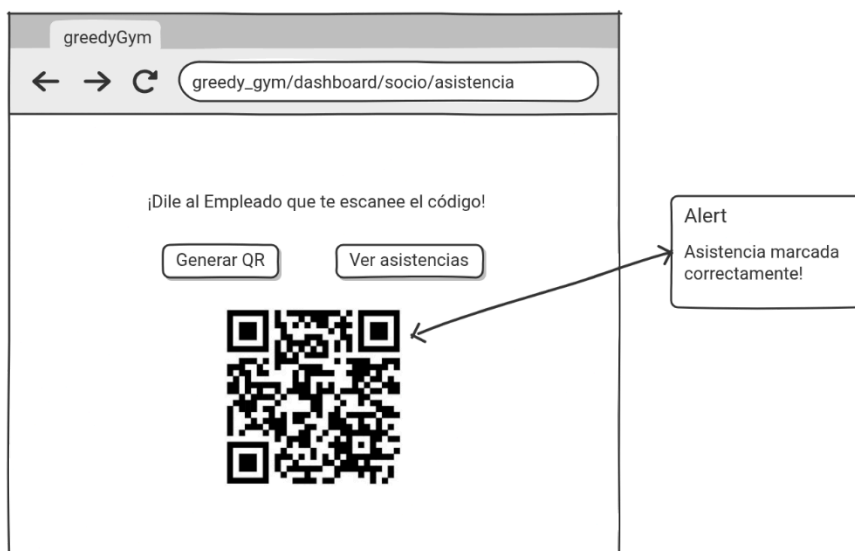
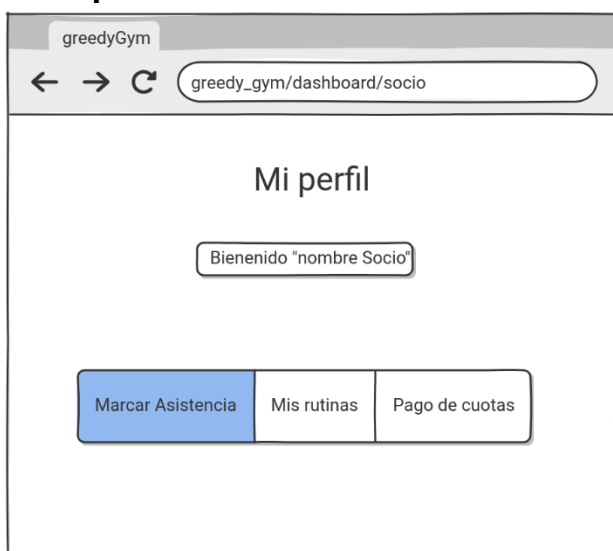
Cada vez que un socio ingrese al gimnasio, se podrá marcar la asistencia a través de un QR. Ahora los socios contarán con una nueva pantalla en su perfil desde la cual podrán marcar su asistencia cada vez que asistan al gimnasio.

Modificación del diagrama de clases

En el siguiente diagrama se representa la nueva clase asistencia, en donde se relaciona con un socio y con un empleado, ya que el empleado deberá escanear el QR que genere el socio



Prototipado



Porcentaje avance del software

Al momento de la presentación del integrador se tiene implementado aproximadamente un 80% del total. El 20% restante corresponde al ABM de rutina junto con sus relaciones con el resto de entidades.

Anexo

Repositorio del proyecto: https://github.com/r-baggioli/ingSoftII/tree/main/greedy_gym

Link a los diagramas:

https://drive.google.com/file/d/1CjeiFq0IS1zihNcNypZ6mmAb-zR_ixLS/view?usp=sharing

Link al diagrama de clases principal en formato pdf:

https://drive.google.com/file/d/1FrMpOX_JYFScvMbqNTU6-4gDwy4C6vKf/view?usp=drive_link

Link a la presentación:

<https://www.canva.com/design/DAGzzu1zWL0/zMLZMokWNtG9GBCdyRle6g/edit>